

Allowing Anonymous Structs as Return Values

Document: P0222R0
Date: 2016-02-05
Project: ISO/IEC JTC1 SC22 WG21 Programming Language C++
Audience: Evolution Working Group
Author: Matthew Woehlke (mwoehlke.floss@gmail.com)

Abstract

This proposal recommends the relaxing of [dcl.fct]/11; specifically, the prohibition of defining (anonymous) types as return values. This proposal is considered contingent on [P0224](#), as obvious problems otherwise arise.

(Note: references made to the existing draft standard are made against [N4567](#).)

Contents

- [Abstract](#)
- [Rationale](#)
- [Proposal](#)
- [Proposed Wording](#)
- [Interactions](#)
- [Implementation and Existing Practice](#)
- [Discussion](#)
 - [What about defining types in function pointer types?](#)
 - [What about defining types in parameter types?](#)
 - [What about "pass-through" of return values having equivalent types?](#)
 - [Conflicts with future "true" multiple return values?](#)
 - [What about deduced return types?](#)
- [Future Directions](#)
- [Acknowledgments](#)
- [References](#)

Rationale

The concept of multiple return values is well known. At present, however, C++ lacks a good mechanism for implementing the same. `std::tuple` is considered clunky by many and, critically, creates sub-optimal API by virtue of the returned values being unnamed, forcing developers to rely on supplemental documentation to explain their purpose. Aggregates represent an improvement, being self-documenting, but the need to provide external definitions of the same is awkward and, worse, pollutes their corresponding namespace with entities that may be single use. Proposals such as [N4560](#) present a complicated mechanism for providing tagged (self-documenting) tuple-like types, which may be necessary in some cases, but still represent a non-trivial amount of complexity that ideally should not be required.

Proposals such as [P0144](#) in particular represent a significant step toward support of multiple return values as first class citizens. This proposals, along with [P0197](#) and other current directions show an encouraging movement away from the traditional `std::pair` and `std::tuple` towards comparable concepts without requiring the explicit types. (We expect, however, that the standard template library types will remain useful for algorithms where the identity of the elements is unimportant, while it is important to be able to name at least the outer, if not complete, type. In that respect, we hypothesize that we may in the future see the ability to construct a `std::tuple` from any tuple-like, as also suggested in [P0197](#).) On their own, however, these proposals risk exacerbating the problem that this proposal aims to address.

It has been suggested on multiple occasions that the optimal solution to the above issues is to return an anonymous struct. This solves the problems of clutter and self-documentation, but runs afoul of a much worse issue; because the struct is *anonymous*, it can be difficult to impossible to give its name a second time in order to separate the declaration and definition of the function that wishes to use it.

The discussion on this topic lead to an interesting question, which is addressed in [P0224](#): **why is it necessary to repeat the**

return value at all? If [P0224](#) is accepted, the problem of naming the return type is immediately eliminated, and along with it, a major reason why returning anonymous structs is not currently permitted.

Proposal

The use of `auto` to indicate an *inferred* return type (as proposed by [P0224](#)) provides an optimal solution to the following problem:

```
// foo.h
struct { int id; double value; } foo();
```

How does one now provide an external definition for `foo()`? With [P0224](#), the solution is simple:

```
// foo.cpp
auto foo()
{
    ...
    return { id, value };
}
```

Recent and proposed changes in C++ significantly mitigate the reasons to prohibit an anonymous struct defined as a return type. Constructing the return result is a non-issue, since the type name may now be elided, and the combination of `auto` variable declarations, `decltype`, and [P0224](#), permit implicit naming of the type where necessary. In short, the prohibition ([dcl.fct]/11) against defining types in return type specifications has become largely an artificial and arbitrary restriction which we propose to remove.

We additionally note that this prohibition is already not enforced by at least one major compiler (MSVC), and is enforced sporadically in others (see [What about defining types in function pointer types?](#)).

Proposed Wording

(Proposed changes are specified relative [N4567](#).)

Change [dcl.fct]/11 (8.3.5.11) as follows:

Types shall not be defined in ~~return-or~~ parameter types.

Interactions

Definition of an anonymous class-type as a return value type is currently ill-formed (although not universally enforced by existing major compilers). Accordingly, this change will not affect existing and conforming code, and may cause existing but non-conforming code to become conforming. This proposal does not make any changes to other existing language or library features; while conceivable that some library methods might benefit from the feature, such changes are potentially breaking, and no such changes are proposed at this time.

Implementation and Existing Practice

The proposed feature is at least already partly implemented by MSVC and (to a lesser extend) GCC and ICC. The curious, partial support in GCC and ICC (see [What about defining types in function pointer types?](#)) suggests that the existing prohibition may already be largely artificial, and that removing it would accordingly be a simple matter.

Discussion

What about defining types in function pointer types?

An obvious consequence of relaxing [dcl.fct]/11 is the desire to permit function pointers which return an anonymous struct. For example:

```
// Declare a function pointer type which returns an anonymous struct
```

```

using ReturnsAnonymousStruct = struct { int result; } (*());

// Define a function using the same
int bar(ReturnsAnonymousStruct f) { return ((*f)()).result; }

// Provide a mechanism to obtain the return type of a function
template <typename T> struct ReturnType;

template <typename T, typename... Args>
struct ReturnType<T (*)(Args...)>
{
    using result_t = T;
};

// Declare a function that is a ReturnsAnonymousStruct
ReturnType<ReturnsAnonymousStruct>::result_t foo() { return {0}; }

// Use the function
int main()
{
    return bar(&foo);
}

```

It is our opinion that the proposed changes are sufficient to allow the above. (In fact, this example is already accepted by both GCC and ICC, although it is rejected by clang per [dcl.fct]/11.) Accordingly, we feel that this proposal should be understood as intending to allow the above example and that additional wording changes to specify this behavior are not required at this time.

What about defining types in parameter types?

An obvious follow-on question is, should we also lift the prohibition against types defined in parameter specifications? There have been suggestions floated to implement the much requested named parameters in something like this manner. However, there are significant (in our opinion) reasons to not address this, at least initially. First, it is widely contested that this is not an optimal solution to the problem (named parameters) in the first place. Second, it depends on named initializers, which is an area of ongoing work. Third, this proposal works largely because C++ forbids overloading on return type, which may be leveraged to eliminate any ambiguity as to the deduction of the actual type of `auto`; this is not the case for parameters, and so permitting `auto` as a parameter type specifier would quickly run into issues that can be avoided for the return type case.

While we do not wish to categorically rule out future changes in this direction, we feel that it is not appropriate for this proposal to attempt to address these issues.

What about "pass-through" of return values having equivalent types?

Another question that has come up is if something like this should be allowed:

```

struct { int result; } foo() { ... }
struct { int result; } bar()
{
    return foo();
}

```

Specifically, others have expressed an interest in treating layout-compatible types as equivalent (or at least, implicitly convertible), particularly in the context of return values as in the above example.

Under the current rules (plus relaxed [dcl.fct]/11), these two definitions have different return types which are not convertible. It is our opinion that the rules making these types different are in fact correct and desirable, and this proposal specifically does *not* include any changes which would make the types compatible. We would, however, encourage a future (orthogonal) proposal which would allow something like this:

```

struct { int result; } bar()
{
    // The use of '...' here implies that the compiler stores the result of
    // 'foo()' in a temporary, which is unpacked into a parameter pack and then
    // expanded into an expression list which is used to form an initializer
    // list which in turn forms the return value of 'bar'. This syntax should
    // be taken as illustrative only; we do not anticipate that this would be
    // the exact syntax used should such a feature be added.
    return { foo()... };
}

```

Conflicts with future "true" multiple return values?

There has been some discussion of "true" multiple return values, in particular with respect to RVO and similar issues. No doubt unpacking, if accepted, will play a part. A point that bears consideration is if moving down the path of using anonymous (or not) structs for multiple return values will "paint us into a corner" where future optimization potential is prematurely eliminated.

It is our hope that these issues can be addressed with existing compound types (which will have further reaching benefit), and that it is accordingly not necessary to hold back the features here proposed in the hope of something better coming along. As is often said, perfect is the enemy of good.

What about deduced return types?

This feature is not compatible with deduced return types at this time. If designated initializers are ever accepted, it might be possible to lift this restriction:

```
auto foo()
{
    return { .x = 3, .y = 2 }; // deduce: struct { int x, y; }
}
```

However, we have reservations about allowing this, and do not at this time propose that this example would be well-formed.

Future Directions

In the [Discussion](#) section above, we presented a utility for extracting the return type from a function pointer type. The facility as presented has significant limitations; namely, it does not work on member functions and the several variations (e.g. CV-qualification) which apply to the same. We do not here propose a standard library implementation of this facility, which presumably would cover these cases, however there is room to imagine that such a facility could be useful, especially if the proposals we present here are adopted. (David Krauss points out that `std::reference_wrapper` can be used to similar effect... on *some* compilers. However, imperfect portability and the disparity between intended function and use for this result suggest that this is not the optimal facility for the problem.)

Another consideration that seems likely to come up is if we should further simplify the syntax for returning multiple values (conceivably, this could apply to both anonymous structs and to `std::pair` / `std::tuple`). Some have suggested allowing that the `struct` keyword may be omitted. In light of [P0151](#), we can conceive that allowing the syntax `<int x, double y> foo()` might be interesting. At this time, we prefer to focus on the feature here presented rather than risk overextending the reach of this proposal. However, if this proposal is accepted, it represents an obvious first step to considering such features in the future.

Acknowledgments

We wish to thank everyone on the `std-proposals` forum, especially Bengt Gustafsson and Tim Song, for their valuable feedback and insights.

References

- [N4567](#) Working Draft, Standard for Programming Language C++
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4567.pdf>
- [N4560](#) Extensions for Ranges
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4560.pdf>
- [P0144](#) Structured Bindings
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/p0144r0.pdf>
- [P0151](#) Proposal of Multi-Declarators (aka Structured Bindings)
<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/p0151r0.pdf>
- [P0197](#) Default Tuple-like Access

<http://wg21.link/p0197>

- [P0224](#) Implicit Return Type

<http://wg21.link/p0224>